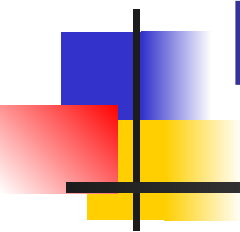
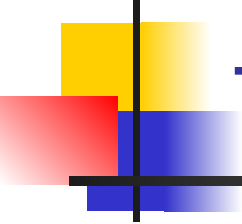


Topics

- 1) Simplifying CFGs, Normal forms
- 2) Pumping lemma for CFLs
- 3) Closure and decision properties of CFLs



How to “simplify” CFGs?



Three ways to simplify/clean a CFG

(clean)

1. Eliminate *useless symbols*

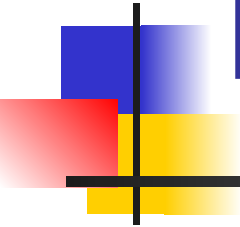
(simplify)

2. Eliminate ϵ -productions

$$A \not\Rightarrow \epsilon$$

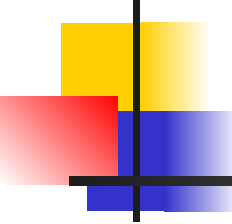
3. Eliminate *unit productions*

$$A \not\Rightarrow B$$



Eliminating useless symbols

Grammar cleanup



Eliminating *useless symbols*

A symbol X is reachable if there exists:

- $S \xrightarrow{*} \alpha X \beta$

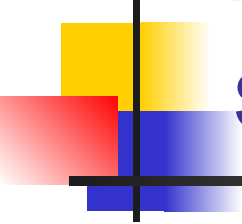
A symbol X is generating if there exists:

- $X \xrightarrow{*} w,$
 - for some $w \in T^*$

For a symbol X to be “useful”, it has to be both reachable *and* generating

- $S \xrightarrow{*} \alpha X \beta \xrightarrow{*} w', \text{ for some } w' \in T^*$

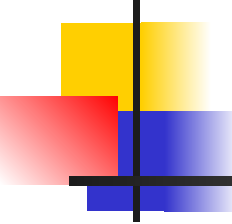
reachable generating



Algorithm to detect useless symbols

1. First, eliminate all symbols that are *not* generating
2. Next, eliminate all symbols that are *not* reachable

Is the order of these steps important,
or can we switch?



Example: Useless symbols

- $S \rightarrow AB \mid a$
- $A \rightarrow b$

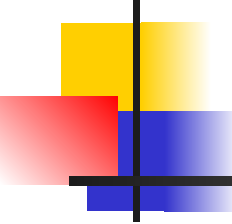
1. A, S are generating
2. B is *not generating* (and therefore B is useless)
3. $\Rightarrow$ Eliminating B ... (i.e., remove all productions that involve B)
 1. $S \rightarrow a$
 2. $A \rightarrow b$
4. Now, A is *not reachable* and therefore is useless

5. Simplified G :

1. $S \rightarrow a$

What would happen if you reverse the order:
i.e., test reachability before generating?

Will fail to remove:
 $A \rightarrow b$


$$X \square^* w$$

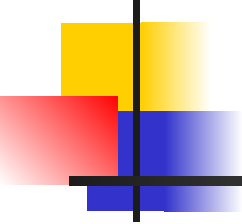
Algorithm to find all generating symbols

- Given: $G=(V,T,P,S)$
- Basis:
 - Every symbol in T is obviously generating.
- Induction:
 - Suppose for a production $A \square \alpha$, where α is generating
 - Then, A is also generating

$$S \Rightarrow^* \alpha X \beta$$

Algorithm to find all reachable symbols

- Given: $G=(V,T,P,S)$
- Basis:
 - S is obviously reachable (from itself)
- Induction:
 - Suppose for a production $A \Rightarrow \alpha_1 \alpha_2 \dots \alpha_k$, where A is reachable
 - Then, all symbols on the right hand side, $\{\alpha_1, \alpha_2, \dots, \alpha_k\}$ are also reachable.

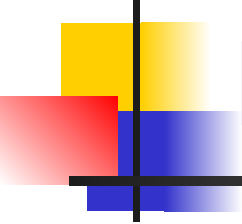


$A \rightarrow 012 \mid X122$

$X \rightarrow X2 \mid 0Y2$

$Y \rightarrow 1Y1 \mid X2$

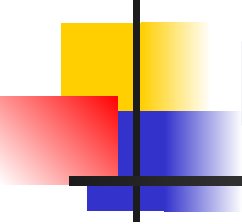
$Z \rightarrow Z1 \mid 2$



Example

Remove useless symbol from the given CFG.

1. $S \rightarrow aB \mid bX$
2. $A \rightarrow BAd \mid bSX \mid a$
3. $B \rightarrow aSB \mid bBX$
4. $X \rightarrow SBD \mid aBX \mid ad$
5. Solution $S \rightarrow bX$
 $X \rightarrow ad$



Example 2

$A \rightarrow 012 \mid X122$

$X \rightarrow X2 \mid 0Y2$

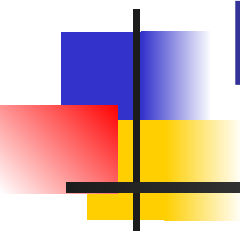
$Y \rightarrow 1Y1 \mid X2$

$Z \rightarrow Z1 \mid 2$

Solution

$A \rightarrow 012$

$Z \rightarrow Z1 \mid 2$ (This should be included in Simplified grammar??)



Eliminating ϵ -productions

$A \Rightarrow \epsilon$

X

What's the point of removing ϵ -productions?

$A \Rightarrow \epsilon$

Eliminating ϵ -productions

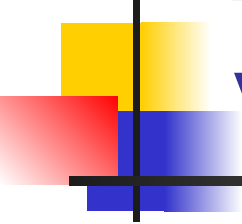
Caveat: It is *not* possible to eliminate ϵ -productions for languages which include ϵ in their word set

So we will target the grammar for the rest of the language

Theorem: If $G=(V,T,P,S)$ is a CFG for a language L , then $L \setminus \{\epsilon\}$ has a CFG without ϵ -productions

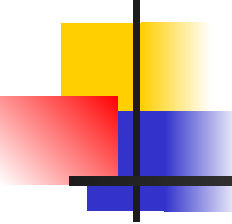
Definition: A is “nullable” if $A \Rightarrow^* \epsilon$

- If A is nullable, then any production of the form “ $B \Rightarrow CAD$ ” can be simulated by:
 - $B \Rightarrow CD \mid CAD$
 - This can allow us to remove ϵ transitions for A



Algorithm to detect all nullable variables

- Basis:
 - If $A \rightarrow \varepsilon$ is a production in G , then A is nullable
(note: A can still have other productions)
- Induction:
 - If there is a production $B \rightarrow C_1 C_2 \dots C_k$, where *every* C_i is nullable, then B is also nullable

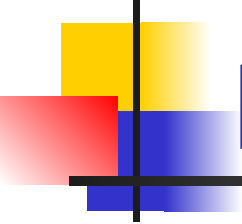


Eliminating ϵ -productions

Given: $G=(V,T,P,S)$

Algorithm:

1. Detect all nullable variables in G
2. Then construct $G_1=(V,T,P_1,S)$ as follows:
 - i. For each production of the form: $A \rightarrow X_1X_2\dots X_k$, where $k \geq 1$, suppose m out of the k X_i 's are nullable symbols
 - ii. Then G_1 will have 2^m versions for this production
 - i. i.e, all combinations where each X_i is either present or absent
 - iii. Alternatively, if a production is of the form: $A \rightarrow \epsilon$, then remove it



Algorithm of removing null production

- Step 1: To remove $A \rightarrow \epsilon$, look for all production whose right side contain A .
- Step 2: Replace each occurrence of A in each of these productions with ϵ .
- Step 3: Add resultant production to Grammar.

Example: Eliminating ϵ -productions

- Let L be the language represented by the following CFG G :

- i. $S \rightarrow AB$
- ii. $A \rightarrow aAA \mid \epsilon$
- iii. $B \rightarrow bBB \mid \epsilon$

Simplified
grammar

Goal: To construct G_1 , which is the grammar for $L - \{\epsilon\}$

- Nullable symbols: $\{A, B\}$
- G_1 can be constructed from G as follows:
 - $B \rightarrow b \mid bB \mid bB \mid bBB$
 - $\Rightarrow B \rightarrow b \mid bB \mid bBB$
 - Similarly, $A \rightarrow a \mid aA \mid aAA$
 - Similarly, $S \rightarrow A \mid B \mid AB$
- Note: $L(G) = L(G_1) \cup \{\epsilon\}$

G_1 :

- $S \rightarrow A \mid B \mid AB$
- $A \rightarrow a \mid aA \mid aAA$
- $B \rightarrow b \mid bB \mid bBB$

+

- $S \rightarrow \epsilon$

Eliminating unit productions

A $\Rightarrow$ B

X

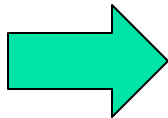
← B has to be a variable

What's the point of removing unit transitions ?

Will save #substitutions

E.g.,

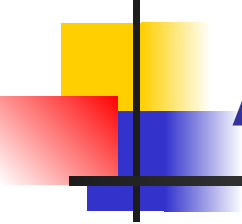
A $\Rightarrow$ B | ...
B $\Rightarrow$ C | ...
C $\Rightarrow$ D | ...
D $\Rightarrow$ xxx | yyy | zzz



A $\Rightarrow$ xxx | yyy | zzz | ...
B $\Rightarrow$ xxx | yyy | zzz | ...
C $\Rightarrow$ xxx | yyy | zzz | ...
D $\Rightarrow$ xxx | yyy | zzz

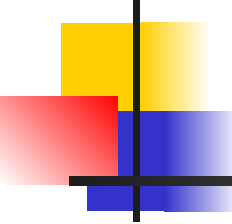
before

after



Algorithm of unit production

- Step 1: To remove $A \rightarrow B$ add production $A \rightarrow x$ to grammar rule whenever $B \rightarrow x$ occur in grammar where x is terminal, x can be null.
- Step 2: Remove $A \rightarrow B$ from grammar
- Step 3: Repeat from step 1 until all unit production are removed.



Example of unit production

$S \rightarrow XY$

$X \rightarrow a$

$Y \rightarrow Z \mid b$

$Z \rightarrow M$

$M \rightarrow N$

$N \rightarrow a$

Unit productions are $Y \rightarrow Z, Z \rightarrow M, M \rightarrow N$

Step1 : To remove $M \rightarrow N$ since $s N \rightarrow a$ we add $M \rightarrow a$ in grammar.

Step 2 : Remove this production rule .

Step 3: Repeat this for all unit production

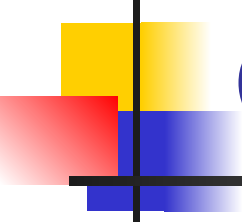
New grammar will be

$S \rightarrow XY, X \rightarrow a, Y \rightarrow Z \mid b, Z \rightarrow M, M \rightarrow a, N \rightarrow a$

Step1 : To remove $Z \rightarrow M$ since $s M \rightarrow a$ we add $Z \rightarrow a$ in grammar.

Step 2 : Remove this production rule .

Step 3: Repeat this for all unit production .



Conti...

New grammar will be

$S \rightarrow XY, X \rightarrow a, Y \rightarrow Z \mid b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$

Step1 : To remove $Y \rightarrow Z$ since $Z \rightarrow a$ we add $Y \rightarrow a$ in grammar.

Step 2 : Remove this production rule .

Step 3: Repeat this for all unit production .

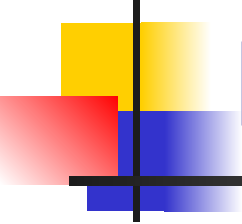
New grammar will be

$S \rightarrow XY, X \rightarrow a, Y \rightarrow a \mid b, Z \rightarrow a, M \rightarrow a, N \rightarrow a$

$$A \Rightarrow B$$

Eliminating unit productions

- Unit production is one which is of the form $A \Rightarrow B$, where both A & B are variables
- E.g.,
 1. $E \Rightarrow T \mid E+T$
 2. $T \Rightarrow F \mid T^*F$
 3. $F \Rightarrow I \mid (E)$
 4. $I \Rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
- How to eliminate unit productions?
 - Replace $E \Rightarrow T$ with $E \Rightarrow F \mid T^*F$
 - Then, upon recursive application wherever there is a unit production:
 - $E \Rightarrow F \mid T^*F \mid E+T$ (substituting for T)
 - $E \Rightarrow I \mid (E) \mid T^*F \mid E+T$ (substituting for F)
 - $E \Rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \mid (E) \mid T^*F \mid E+T$ (substituting for I)
 - Now, E has no unit productions
 - Similarly, eliminate for the remainder of the unit productions

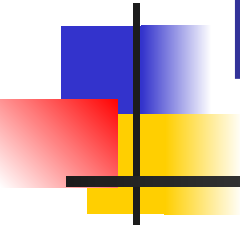


Putting all this together...

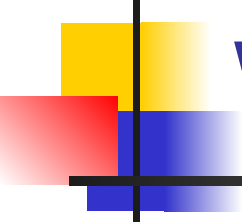
- Theorem: If G is a CFG for a language that contains at least one string other than ε , then there is another CFG G_1 , such that $L(G_1) = L(G) - \varepsilon$, and G_1 has:
 - no ε -productions
 - no unit productions
 - no useless symbols
- Algorithm:
 - Step 1) eliminate ε -productions
 - Step 2) eliminate unit productions
 - Step 3) eliminate useless symbols

Again,
the order is
important!

Why?

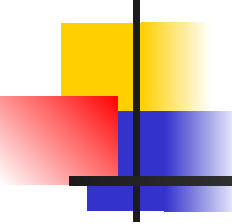


Normal Forms



Why normal forms?

- If all productions of the grammar could be expressed in the same form(s), then:
 - a. It becomes easy to design algorithms that use the grammar
 - b. It becomes easy to show proofs and properties



Chomsky Normal Form (CNF)

Let G be a CFG for some $L - \{\epsilon\}$

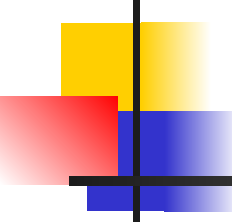
Definition:

G is said to be in **Chomsky Normal Form** if all its productions are in one of the following two forms:

i. $A \rightarrow BC$ where A, B, C are variables, or

ii. $A \rightarrow a$ where a is a terminal

- G has no useless symbols
- G has no unit productions
- G has no ϵ -productions



CNF checklist

Is this grammar in CNF?

G_4 :

1. $E \rightarrow E+T \mid T^*F \mid (E) \mid Ia \mid Ib \mid I0 \mid I1$
2. $T \rightarrow T^*F \mid (E) \mid Ia \mid Ib \mid I0 \mid I1$
3. $F \rightarrow (E) \mid Ia \mid Ib \mid I0 \mid I1$
4. $I \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$

Checklist:

- G has no ϵ -productions ✓
- G has no unit productions ✓
- G has no useless symbols ✓
- But...
 - the normal form for productions is violated



So, the grammar is not in CNF

How to convert a G into CNF?

- Assumption: G has no ϵ -productions, unit productions or useless symbols
- 1) For every terminal a that appears in the body of a production:
 - i. create a unique variable, say X_a , with a production $X_a \rightarrow a$, and
 - ii. replace all other instances of a in G by X_a
- 2) Now, all productions will be in one of the following two forms:
 - $A \rightarrow B_1 B_2 \dots B_k$ ($k \geq 3$) or $A \rightarrow a$
- 4) Replace each production of the form $A \rightarrow B_1 B_2 B_3 \dots B_k$ by:

$$\begin{array}{c}
 \xleftarrow{C_2} \quad \xrightarrow{C_2} \\
 B_2 \quad B_3 \quad \dots \quad B_k \\
 \xleftarrow{C_1} \quad \xrightarrow{C_1} \\
 B_1 \quad B_2 \quad B_3 \quad \dots \quad B_k
 \end{array}$$

and so on...

 - $A \rightarrow B_1 C_1 \quad C_1 \rightarrow B_2 C_2 \quad \dots \quad C_{k-3} \rightarrow B_{k-2} C_{k-2} \quad C_{k-2} \rightarrow B_{k-1} B_k$

Example #1

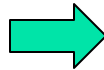
G:

$S \Rightarrow AS \mid BABC$

$A \Rightarrow A1 \mid 0A1 \mid 01$

$B \Rightarrow 0B \mid 0$

$C \Rightarrow 1C \mid 1$



G in CNF:

$X_0 \Rightarrow 0$

$X_1 \Rightarrow 1$

$S \Rightarrow AS \mid BY_1$

$Y_1 \Rightarrow$

AY_2

$A \Rightarrow BC \mid X_0 Y_3 \mid X_0 X_1$

$Y_3 \Rightarrow AX_1$

$B \Rightarrow X_0 B \mid 0$

$C \Rightarrow X_1 C \mid 1$

All productions are of the form: $A \Rightarrow BC$ or $A \Rightarrow a$

Example #2

G:

1. $E \sqsubseteq E+T \mid T^*F \mid (E) \mid I_a \mid I_b \mid I_0 \mid I_1$
2. $T \sqsubseteq T^*F \mid (E) \mid I_a \mid I_b \mid I_0 \mid I_1$
3. $F \sqsubseteq (E) \mid I_a \mid I_b \mid I_0 \mid I_1$
4. $I \sqsubseteq a \mid b \mid I_a \mid I_b \mid I_0 \mid I_1$

Step (1)

1. $E \sqsubseteq EX_+T \mid TX_*F \mid X(EX_) \mid IX_a \mid IX_b \mid IX_0 \mid IX_1$
2. $T \sqsubseteq TX_*F \mid X(EX_) \mid IX_a \mid IX_b \mid IX_0 \mid IX_1$
3. $F \sqsubseteq X(EX_) \mid IX_a \mid IX_b \mid IX_0 \mid IX_1$
4. $I \sqsubseteq X_a \mid X_b \mid IX_a \mid IX_b \mid IX_0 \mid IX_1$
5. $X_+ \sqsubseteq +$
6. $X_* \sqsubseteq *$
7. $X_+ \sqsubseteq +$
8. $X_ (\sqsubseteq ($
9.

Step (2)

1. $E \sqsubseteq EC_1 \mid TC_2 \mid X(C_3 \mid IX_a \mid IX_b \mid IX_0 \mid IX_1$
2. $C_1 \sqsubseteq X_+T$
3. $C_2 \sqsubseteq X_*F$
4. $C_3 \sqsubseteq EX_)$
5. $T \sqsubseteq \dots\dots\dots$
6.

Languages with ϵ

- For languages that include ϵ ,
 - Write down the rest of grammar in CNF
 - Then add production “ $S \Rightarrow \epsilon$ ” at the end

E.g., consider:

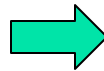
G:

$S \Rightarrow AS \mid BABC$

$A \Rightarrow A1 \mid 0A1 \mid 01 \mid \epsilon$

$B \Rightarrow 0B \mid 0 \mid \epsilon$

$C \Rightarrow 1C \mid 1 \mid \epsilon$



G in CNF:

$X_0 \Rightarrow 0$

$X_1 \Rightarrow 1$

$S \Rightarrow AS \mid BY_1 \mid \epsilon$

$Y_1 \Rightarrow AY_2$

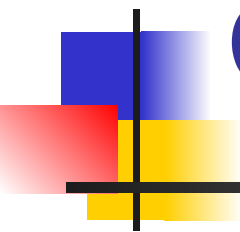
$Y_2 \Rightarrow BC$

$A \Rightarrow AX_1 \mid X_0Y_3 \mid X_0X_1$

$Y_3 \Rightarrow AX_1$

$B \Rightarrow X_0B \mid 0$

$C \Rightarrow X_1C \mid 1$



CFL Closure Properties
